

ระบบแจ้งซ่อมออนไลน์

“ศูนย์งานซ่อม”

Online Repair Request System — พัฒนาด้วย Django 6 · ทำงานเป็นทีม 4 คนผ่าน Git & GitHub

ผู้ออกแบบฐานข้อมูล

ชื่อ-นามสกุล

วริทธิ์พล ไพรสิงห์

รหัสนักศึกษา

feature/models

ผู้ทำระบบสมาชิก

ชื่อ-นามสกุล

ปณณวิชญ์ ราชเสนา

รหัสนักศึกษา

feature/auth

ผู้ทำ workflow

ชื่อ-นามสกุล

สุวรรณี ทองบ่อ

รหัสนักศึกษา

feature/workflow

ผู้ทำ UI/UX

ชื่อ-นามสกุล

ปรีตธีการส สมพล

รหัสนักศึกษา

feature/ui

โครงการนี้คืออะไร – สรุปในหน้าเดียว

เว็บแอปพลิเคชันที่ทำให้ “การแจ้งซ่อม” ทั้งกระบวนการอยู่ในระบบเดียว ตั้งแต่ผู้ใช้กรอกใบแจ้ง → ผู้ดูแลตรวจและมอบหมายช่าง → ช่างซ่อมและบันทึกผล → ผู้ดูแลตรวจรับ → ผู้แจ้งให้คะแนน โดยทุกฝ่ายเห็นสถานะเดียวกันตลอดเวลา



จุดขายของโครงการ: ไม่ได้หยุดที่ “เขียนโค้ดให้ทำงานได้” แต่ทำครบตามหลัก DevOps — แยก branch ต่อคน, เปิด Pull Request ให้เพื่อนรีวิว, มี GitHub Actions ทดสอบอัตโนมัติทุกครั้ง push และบันทึกทุกเวอร์ชันไว้ใน CHANGELOG

ปัญหาของการแจ้งซ่อมแบบเดิม

แบบเดิม

- ▶ แจ้งด้วยปากเปล่า เขียนใส่กระดาษ หรือส่งข้อความในไลน์กลุ่ม — **ใบแจ้งหายและตกหล่น**
- ▶ ผู้แจ้ง **ไม่รู้ว่างานถึงขั้นไหน** ต้องเดินไปถามเองซ้ำ ๆ
- ▶ ไม่ชัดเจนว่า **ช่างคนไหนรับผิดชอบ** งานถูกโยนไปมา
- ▶ ไม่มีหลักฐานสภาพก่อน-หลังซ่อม เทียบกันภายหลัง
- ▶ ผู้บริหาร **ไม่มีข้อมูลสรุป** ว่าเดือนนี้ซ่อมอะไรไปกี่งาน ช่างคนไหนงานล้น

หลังใช้ระบบนี้

- ▶ ใบแจ้งซ่อมทุกใบอยู่ในฐานข้อมูลเดียว **มีเลขที่ใบกำกับ** ค้นย้อนหลังได้
- ▶ ผู้แจ้งเห็น **ไทม์ไลน์สถานะ** ของใบตัวเองได้ตลอดเวลา
- ▶ ผู้ดูแลกดมอบหมายช่าง ระบบ **บันทึกชื่อช่างและเวลา** ให้อัตโนมัติ
- ▶ แบบ **รูปก่อนซ่อม (ผู้แจ้ง) และรูปหลังซ่อม (ช่าง)** เป็นหลักฐานในใบเดียวกัน
- ▶ มี **หน้ารายงานสรุป** ตามสถานะ ประเภทงาน ภาระงานช่าง และคะแนนความพึงพอใจเฉลี่ย

วัตถุประสงค์และขอบเขตของโครงการ

วัตถุประสงค์

- ▶ พัฒนาเว็บแอปที่รองรับกระบวนการแจ้งซ่อม**ครบวงจร** ตั้งแต่แจ้งจนถึงประเมินผล
- ▶ ออกแบบฐานข้อมูลเชิงสัมพันธ์ให้เก็บ**ประวัติงานซ่อมย้อนหลัง**ได้จริง
- ▶ ควบคุม**สิทธิ์การเข้าถึงตามบทบาท** ให้แต่ละคนเห็นเฉพาะข้อมูลที่ตนควรเห็น
- ▶ ฝึก**การทำงานเป็นทีมด้วย Git/GitHub** — branch, Pull Request, code review
- ▶ นำหลัก **DevOps** มาใช้จริง: ทดสอบอัตโนมัติด้วย CI ทุกครั้งที่ส่งโค้ด

ขอบเขต — ทำอะไร / ยังไม่ได้ทำ

อยู่ในขอบเขต

- ระบบสมาชิก 3 บทบาท + สมาชิกสมาชิกเอง
- แจ้งซ่อม แบบรูป แก้ไข ยกเลิก ติดตามสถานะ
- มอบหมายช่าง · ติ๊กกลับ · ตรวจสอบ · ประเมินผล
- ห้องสนทนาต่อใบแจ้งซ่อม + แนบไฟล์ + แจ้งเตือนข้อความใหม่
- คอนโซลจัดการและรายงานสรุปสำหรับผู้ดูแล

ยังไม่อยู่ในขอบเขต

- ยังใช้ SQLite (เหมาะกับการเรียน) ยังไม่ย้ายขึ้น PostgreSQL
- ยังไม่ deploy ขึ้นเซิร์ฟเวอร์จริง — รันบนเครื่อง **127.0.0.1:8000**
- ยังไม่มีแจ้งเตือนทางอีเมล/LINE และยังไม่มีแอปมือถือ (แต่นำเว็บรองรับจอสือแล้ว)

เทคโนโลยีที่ใช้ และเหตุผลที่เลือก

ส่วนของระบบ	เทคโนโลยี	ทำไมถึงเลือกตัวนี้
ภาษา & เฟรมเวิร์ก	Python 3.12 · Django 6.0.7	มี ORM, ระบบ auth, ระบบ admin และระบบ migration มาให้ในตัว จึงโฟกัสที่ตรรกะงานซ่อมได้เต็มที่
ฐานข้อมูล	SQLite (ไฟล์ db.sqlite3)	ไม่ต้องติดตั้งเซิร์ฟเวอร์ฐานข้อมูล เพื่อนในทีมทุกคน clone แล้วรันได้ทันที และย้ายไป PostgreSQL ภายหลังได้โดยไม่ต้องแก้ไขโค้ด
หน้าเว็บ	Django Template · Bootstrap 5.3 · Bootstrap Icons	ได้ระบบ grid ที่รองรับมือถืออยู่แล้ว และเขียน CSS เพิ่มเองเป็น design system กับได้
รูปภาพ	Pillow 12.3.0	ใช้ตรวจว่าไฟล์ที่อัปโหลด “เป็นรูปจริง” ไม่ใช่แค่เปลี่ยนนามสกุล — เป็นเรื่องความปลอดภัย
ตัวอักษร	IBM Plex Sans Thai · Sarabun	ฟอนต์ไทยอ่านง่าย เก็บไว้ในโปรเจกต์เอง ไม่พึ่ง Google Fonts — เปิดใช้งานได้แม้ห้องเรียนไม่มีอินเทอร์เน็ต
DevOps	Git · GitHub · GitHub Actions	แยก branch ต่อกัน เปิด PR ให้เพื่อนรีวิว และรันทดสอบอัตโนมัติก่อน merge ทุกครั้ง
การทดสอบ	Django TestCase (unittest)	ทดสอบได้ทั้งโมเดล สิกซ์ และการกดปุ่มจริงผ่าน test client โดยไม่ต้องลงเครื่องมือเพิ่ม

หลักการที่ยึดตลอดโครงการ: ไฟล์ Bootstrap ไอคอน และฟอนต์ทั้งหมดถูกดาวน์โหลดมาเก็บไว้ใน `static/vendor/` ทำให้ระบบ **เปิดใช้งานได้แบบออฟไลน์ 100%** ตอนนำเสนอจึงไม่ต้องลุ้นว่าเน็ตห้องเรียนจะล่มหรือไม่

ผู้ใช้ 3 บทบาท — ใครทำอะไรได้บ้าง

ระบบไม่ได้แค่ “ซ่อนปุ่ม” ตามบทบาท แต่ตรวจสอบสิทธิ์ซ้ำที่ฝั่งเซิร์ฟเวอร์ทุกครั้ง ถ้าผู้ใช้พยายามยิงคำสั่งตรงไปที่ URL ที่ไม่มีสิทธิ์ ระบบจะตอบกลับ 403 ทันที

ผู้แจ้งซ่อม

reporter — สมัครเองได้จากหน้าเว็บ

- สมัครสมาชิก / เข้าสู่ระบบ
- สร้างใบแจ้งซ่อม + แบนรูปได้สูงสุด 5 รูป
- แก้ไขหรือยกเลิกใบที่ยัง “รอตรวจสอบ” หรือถูก “ตีกลับ”
- ติดตามไทม์ไลน์สถานะของใบตัวเอง
- คุยกับผู้ดูแล/ช่างในห้องสนทนาของใบนั้น
- ให้คะแนนความพึงพอใจ 1–5 ดาว เมื่องานเสร็จ

เห็นได้เฉพาะใบแจ้งซ่อมของตัวเองเท่านั้น

ผู้ดูแลระบบ

admin — สร้างโดยผู้ดูแลเดิมเท่านั้น

- แดชบอร์ดภาพรวม + คิวงานที่ต้องลงมือ
- ตรวจสอบใบแจ้งซ่อม แล้วมอบหมายช่าง หรือตีกลับพร้อมเหตุผล
- ตรวจสอบรับงาน: ผ่าน หรือ ส่งกลับให้ช่างแก้ไข
- คอนโซลจัดการ `/manage/` — คิวงาน การงานช่าง เพิ่มประเภทงาน
- รายงานสรุป `/report/`
- เขียน “หมายเหตุภายใน” ที่ผู้แจ้งมองไม่เห็น

เห็นใบแจ้งซ่อมทุกใบในระบบ

ช่างซ่อม

technician — สร้างโดยผู้ดูแล

- ดูเฉพาะงานที่ถูกมอบหมายให้ตัวเอง
- กด “รับงาน” เพื่อเริ่มดำเนินการ
- บันทึกรายละเอียดการซ่อม + แบนรูปงานที่เสร็จ
- แจ้งงานเสร็จเพื่อส่งให้ผู้ดูแลตรวจสอบ
- สอบถามรายละเอียดเพิ่มในห้องสนทนา

เปิดใบที่ไม่ได้มอบหมายให้ตนไม่ได้ แม้จะรู้เลขที่ใบ

โครงสร้างฐานข้อมูล 9 ตาราง

ตารางและฟิลด์สำคัญ

User ผู้ใช้ PK id username role 3 ค่า phone	RepairRequest ใบแจ้งซ่อม PK id FK reporter FK category title · detail status 7 ค่า location admin_note created_at	Category ประเภทงาน PK id name UNIQUE
Assignment มอบหมายงาน PK id FK request FK technician assigned_date work_detail work_date	RepairImage รูปประกอบ PK id FK request kind report/work image · caption	Rating ประเมินผล PK id 1:1 request score 1-5 comment
Comment ข้อความสนทนา PK id FK request FK author kind user/system is_internal body	CommentAttachment ไฟล์แนบ PK id FK comment file original_name	ChatRead สถานะการอ่าน PK id FK user FK request last_read_at UNIQUE (user, request)

ความสัมพันธ์ 11 เส้น

User ผู้แจ้ง	1:M	RepairRequest
Category	1:M	RepairRequest
RepairRequest	1:M	Assignment
User ช่าง	1:M	Assignment
RepairRequest	1:M	RepairImage
RepairRequest	1:M	Comment
User ผู้เขียน	1:M	Comment
Comment	1:M	CommentAttachment
User	1:M	ChatRead
RepairRequest	1:M	ChatRead
RepairRequest	1:1	Rating

ลบข้อมูลอย่างมีเงื่อนไข: category และ technician ใช้ PROTECT — ลบประเภทงานหรือลบบัญชีช่างทิ้งไม่ได้ เพราะประวัติงานซ่อมจะกำพรว้า ส่วนรูปข้อความ ไฟล์แนบ และผลประเมิน ใช้ CASCADE เพราะไม่มีความหมายถ้าใบแจ้งซ่อมหายไป

วงจรชีวิตของใบแจ้งซ่อม – 7 สถานะ

ทุกการเปลี่ยนสถานะมีเงื่อนไขกำกับในโค้ด: ถ้าสถานะปัจจุบันไม่ตรงกับที่กำหนด ระบบจะปฏิเสธและแจ้งเตือน – กดข้ามขั้นตอนไม่ได้ แม้จะยิง URL ตรง ๆ



แบ่งงานอย่างไร – และทำไมแบ่งแบบนี้

ทีมแบ่งงานตาม **ชั้นของสถาปัตยกรรม (layer)** ไม่ใช่แบ่งตามหน้าจอ เพราะทำให้แต่ละคนเป็นเจ้าของไฟล์คนละชุด แก่กับกันน้อยที่สุด และมี Pull Request ของตัวเองที่ตรวจสอบย้อนหลังได้ชัดเจน

#	หน้าที่	ชื่อ-นามสกุล	ไฟล์ที่เป็นเจ้าของ	BRANCH
1	ผู้ออกแบบฐานข้อมูล	วริทธิ์พล ไพรสิงห์	models.py · migrations/ · admin.py · seed_data.py	feature/models
2	ผู้ทำระบบสมาชิก	ปณณวิชญ์ ราชเสนา	decorators.py · SignUpForm · settings.py · registration/	feature/auth
3	ผู้ทำ workflow	สุวรรณี ทองบ่อ	views.py · urls.py · _actions.html · view report / manage	feature/workflow
4	ผู้ทำ UI/UX	ปรีติธรรส สมพล	templates/ · style.css · templatetags/ · report.html / manage.html	feature/ui

งานที่ทีมรับผิดชอบร่วมกัน: ชดทดสอบ 59 เคส — แต่ละคนเขียนเทสต์ของส่วนที่ตัวเองทำ · GitHub Actions · CHANGELOG.md · คู่มือ Git ของทีม

และทุกคนเห็นงานของกันและกัน: ทุก branch เข้าสู่ main ผ่าน Pull Request ที่มีเพื่อนอย่างน้อย 1 คนกด Approve และต้องผ่าน GitHub Actions ก่อน — ไม่มีใคร push ตรงเข้า main

1 ผู้ออกแบบฐานข้อมูล

วริทธิ์พล ไพรสิงห์ branch `feature/models`

repairs/models.py · 431 บรรทัด migrations/ 6 ไฟล์ · 261 บรรทัด repairs/admin.py · 216 บรรทัด seed_data.py · 119 บรรทัด เทสต์: ModelTests 6 เคส

สิ่งที่ทำ

- ▶ ออกแบบตารางทั้งหมด **9 ตาราง** ด้วย Django ORM — เขียนเป็นคลาส Python แล้วให้ระบบสร้างตาราง SQL ให้ (ในฐานข้อมูลจริงเป็น 11 ตาราง เพราะ Django สร้างตารางเชื่อมสิทธิ์เพิ่มอีก 2)
- ▶ ทำ **Custom User Model** ตั้งแต่วันแรก เพิ่มฟิลด์ “บทบาท” และ “เบอร์โทร” เข้าไปในตารางผู้ใช้โดยตรง ทำให้ทั้งระบบเช็คสิทธิ์จากที่เดียว
- ▶ วางความสัมพันธ์ **11 เส้น** (Foreign Key 10 + One-to-One 1) โดยเลือก `on_delete` ที่ละเส้นตามความหมายจริงของงาน
- ▶ ตั้งชื่อภาษาไทยให้ทุกฟิลด์ **ครบ 39 จาก 39 ฟิลด์** — หน้าหลังบ้านจึงเป็นภาษาไทยโดยไม่ต้องแปลทีละหน้า
- ▶ แบ่งการพัฒนาเป็น **6 migration** ไล่ตามฟีเจอร์ที่เพิ่มเข้ามา ฐานข้อมูลโตขึ้นโดยข้อมูลเดิมไม่หาย
- ▶ ปรับหน้า Django admin **6 หน้า + inline 4 ตัว** พร้อมป้ายสถานะสี รูปย่อ ตัวกรองและช่องค้นหา

ตัวเลขและจุดที่ควรถูกถาม

9
โมเดล
(11 ตารางจริง)

11
เส้นความสัมพันธ์
FK 10 · 1:1 หนึ่ง

6
เทสต์ที่ดูแลเอง
ModelTests

ไม่ได้ใส่ **CASCADE** ทั้งหมดแบบมักง่าย — `category` และ `technician` ใช้ `PROTECT` เพราะลบประเภทงานหรือลบบัญชีช่างทิ้งแล้วประวัติงานซ่อมจะกำพรว้า ส่วนรูป ข้อความ ไฟล์แนบ และผลประโยชน์ ใช้ `CASCADE` เพราะไม่มีความหมายถ้าใบแจ้งซ่อมหายไป

ถาม: แล้วทำไม `reporter` ถึงใช้ **CASCADE** ทั้งที่บอกว่าต้องเก็บประวัติ?

ตอบ: เป็นความไม่สมมาตรที่เรารู้ตัวครับ นโยบายที่ตั้งใจไว้คือ **ไม่ลบบัญชีผู้ใช้จริง** แต่ปิดด้วย `is_active=False` แทน ถ้าจะให้รัดกุมกว่านี้ควรเปลี่ยนเป็น `PROTECT` หรือ `SET_NULL` — เป็นสิ่งที่จะแก้ในรอบถัดไป

2 ผู้ทำระบบสมาชิก

ปณณวิชญ์ ราชเสนา branch `feature/auth`

`repairs/decorators.py` · 33 บรรทัด `forms.py` → `SignUpForm` `config/settings.py` (auth) `templates/registration/` · 65 บรรทัด
เทสต์: `AccessControl` + `CommentThread` 13 เคส

สิ่งที่ทำ

- ▶ ออกแบบระบบ **3 บทบาท**ให้อยู่ในไฟล์เดียว (`role`) แล้วทำ property ช่วยเช็คบทบาท 3 ตัว ให้โค้ดที่เหลือเรียกใช้ง่าย
- ▶ หน้าสมัครสมาชิกที่ **กั้นการยกระดับสิทธิ์ตัวเอง 2 ชั้น** — พอร์มไม่มีช่อง `role` ให้กรอก และตอนบันทึกยังเช็คกับเป็น “ผู้แจ้งซ่อม” เสมอ ต่อให้ยิงค่าปลอมมาก็ไม่ผ่าน
- ▶ เขียน decorator `role_required` ตัวเดียว ใช้ซ้ำ **12 จุด** ทั่วระบบ และ **แยกกรณี**ให้ชัด: ยังไม่ล็อกอิน → พาไปหน้า login, บทบาทไม่ถูก → คีน **403 Forbidden**
- ▶ ตรวจสอบสิทธิ์ **2 ระดับ** คือระดับบทบาท (ใครกดปุ่มนี้ได้) และระดับตัวข้อมูลรายใบ (ใบนี้เป็นของใคร / มอบหมายให้ใคร)
- ▶ “หมายเหตุภายใน” ถูกกัน **ทั้งตอนเขียนและตอนอ่าน** — ผู้แจ้งตั้งค่านี้ไม่ได้ และวิธีที่ดึงข้อความก็กรองออกให้ตั้งแต่ต้นทาง ไม่ได้ซ่อนแค่ที่หน้าจอ
- ▶ ใช้ระบบยืนยันตัวตนมาตรฐานของ Django ครบชุด: รหัสผ่าน hash ด้วย **PBKDF2-SHA256 + salt**, `csrf_token` ครบ **15 จุด**, และปุ่มออกจากระบบเป็น **POST** ตามมาตรฐาน Django 6

ตัวเลขและจุดที่ควรถูกถาม

17/18
view ที่มีการด
ตรวจสอบสิทธิ์

15
จุดที่ใช้
csrf_token

13
เทสที่ดูแลเอง
สิทธิ์ + ห้องสนทนา

สิ่งที่พิสูจน์ได้หน้าห้อง: ล็อกอินเป็นผู้แจ้งซ่อมแล้วพิมพ์ `/manage/` ตรง ๆ → ได้ **403** · เปิดใบแจ้งซ่อมของคนอื่น → ถูกแจ้งเตือนพร้อมข้อความเตือน · ช่างเปิดงานที่ไม่ได้มอบหมายให้ตน → เข้าไม่ได้

ถาม: รหัสผ่านเก็บยังไง ปลอดภัยแค่ไหน?

ตอบ: ไม่ได้เก็บรหัสผ่านจริงครับ Django hash ด้วย **PBKDF2-SHA256** พร้อม salt ต่อผู้ใช้และวนหลายแสนรอบ ในฐานข้อมูลจึงเห็นเป็นสตริงยาวที่ย้อนกลับไม่ได้ ส่วนรหัส `1234` เป็นคำสำหรับเดโมเท่านั้น — ระบบตั้ง `MinimumLengthValidator` ไว้ที่ 4 ตัวเพื่อให้สาริตสะดวก ถ้าใช้งานจริงต้องปรับกลับเป็น 8

3 ผู้ทำ workflow (กระบวนการแจ้งซ่อม)

สัวรรณี ทองบ่อ branch `feature/workflow`

repairs/views.py · 652 บรรทัด

repairs/urls.py · 18 เส้นทาง

requests/_actions.html · 113 บรรทัด

view report + manage · 266 บรรทัด

เทสต์: Workflow/ActivityLog/Manage/Unread 25 เคส

สิ่งที่ทำ

- ▶ เขียน **state machine 7 สถานะ** เป็น view 18 ฟังก์ชันบน 18 เส้นทาง URL — เป็นหัวใจที่เชื่อมทุกบทบาทเข้าด้วยกัน
- ▶ ใส่ **ด่านตรวจสอบสถานะ 10 จุด** ก่อนเปลี่ยนสถานะทุกครั้ง กดข้ามขั้นไม่ได้แม้จะยิง URL ตรง ๆ เช่น “ตรวจรับได้เฉพาะงานที่รอตรวจรับ”, “ยกเลิกใบที่ช่างลงมือแล้วไม่ได้”
- ▶ ทำ **ปุ่มติ๊กกลับที่ฉลาด** — ปุ่มเดียวแต่ปลายทางต่างกันตามสถานะ: ตอน “รอตรวจสอบ” ส่งกลับให้ผู้แจ้งแก้ ส่วนตอน “รอตรวจรับ” ส่งกลับให้ช่างแก้
- ▶ ใช้ **transaction.atomic 3 จุด** (มอบหมายช่าง · แจ้งงานเสร็จ · ส่งข้อความพร้อมไฟล์แนบ) เพื่อไม่ให้ข้อมูลค้างครึ่งทางถ้าเกิด error กลางคัน
- ▶ ทำระบบ **บันทึกความคืบหน้าอัตโนมัติ 6 จุด** — ทุกครั้งที่ผู้ดูแลหรือช่างเปลี่ยนสถานะ ระบบโพสต์ข้อความลงห้องสนทนาให้เอง ผู้แจ้งจึงไม่ต้องถาม
- ▶ บังคับปุ่มที่เปลี่ยนข้อมูล **9 จุด** ให้เป็น POST อย่างเดียว และกรอง queryset ตามบทบาทในหน้ารายการ พร้อม **prefetch_related** กันปัญหา N+1

ตัวเลขและจุดที่ควรถูกถาม

10

ด่านตรวจสอบสถานะ
ก่อนเปลี่ยนค่า

9

ปุ่มที่บังคับ
POST เท่านั้น

25

เทสต์ที่ดูแลเอง
มากที่สุดในทีม

ตัวอย่างด่านที่เห็นผลจริง: ใบที่ถูกติ๊กกลับไปแล้ว ปุ่ม “ติ๊กกลับ” จะหายไปจากหน้าจอ และต่อให้ยังคำสั่งตรงก็ไม่เปลี่ยนสถานะ — กันไม่ให้งานตกลงไปในทางตัน มีเทสต์เลือกพฤติกรรมนี้ไว้ 2 เคส

ถาม: ถ้าช่างลาป่วยกลางทาง จะเปลี่ยนช่างยังไง?

ตอบ: ตอนนี้อย่างทำไม่ได้ครับ ระบบยอมให้มอบหมายเฉพาะใบที่สถานะ “รอตรวจสอบ” หรือ “ติ๊กกลับ” เท่านั้น เป็นข้อจำกัดที่เราจัดไว้และอยู่ในแผนพัฒนาต่อ — วิธีแก้คือเปิดให้ผู้ดูแลมอบหมายใหม่ได้ตอน `assigned` / `in_progress` ซึ่งตาราง `Assignment` รองรับอยู่แล้วเพราะเก็บได้หลายแถวต่อหนึ่งใน

4 ผู้ทำ UI/UX

ปรีตธีการร สมพ branch `feature/ui`

templates/ 16 ไฟล์ · 1,284 บรรทัด static/css/style.css · 356 บรรทัด templatetags/repair_extras.py · 68 บรรทัด report.html + manage.html

เทสต์: ImageUpload + ChatAttachment 15 เคส

สิ่งที่ทำ

- ▶ สร้าง **design system** ของตัวเอง — ตัวแปรสี 28 ตัวใน CSS ที่ถูกเรียกใช้ **264 ครั้ง** ทำให้เปลี่ยนสีทั้งเว็บได้จากที่เดียว
- ▶ ทำ **โหมดสว่าง/มืด** ที่จำค่าไว้ใน `localStorage` และตั้งธีมก่อนหน้าเว็บถูกวาด จึงไม่มีอาการจกระพริบขาววบบตอนโหลด (FOUC)
- ▶ Re-skin คอมโพเนนต์ของ Bootstrap (ปุ่ม ฟอรม ตาราง ป้าย แจ้งเตือน) ด้วยกฎ CSS ของตัวเองราว **90 บรรทัด** ที่อ้าง `var()` ทั้งหมด — โดยไม่แตะไฟล์ `bootstrap.min.css` แม้แต่บรรทัดเดียว
- ▶ เลย์เอาต์ **sidebar + แถบบน** ที่พับเป็นเมนูมือถือได้ด้วย CSS เกือบล้วน (breakpoint เดียวที่ 992px)
- ▶ ออกแบบ UX ห้องแชท: **ลากไฟล์มาวาง · Ctrl+V วางรูปจากคลิปบอร์ด ·** ซิปไฟล์พร้อมปุ่มลบ · Enter ส่ง / Shift+Enter ขึ้นบรรทัด — และ**ตรวจ IME ภาษาไทย**ไม่ให้ Enter ส่งข้อความขณะกำลังพิมพ์คำ
- ▶ ทำระบบให้ **ออฟไลน์ได้ 100%** — เก็บ Bootstrap ไอคอน และฟอนต์ไทยไว้ในโปรเจกต์ 35 ไฟล์ ลิงก์ออกอินเทอร์เน็ตในเกมเพลต **0 ลิงก์**
- ▶ ลดโค้ดซ้ำด้วย `{% extends %}` 11 หน้า, `{% include %}` 20 จุด และ templatetag ที่เขียนเอง 3 ตัว (เช่น ไอคอนประเภทงานที่เดาจากชื่อตัวเอง)

ตัวเลขและจุดที่ควรถูกถาม

264

ครั้งที่เรียกใช้
ตัวแปรสีธีม

0

ลิงก์ CDN
ออกอินเทอร์เน็ต

15

เทสต์ที่ดูแลเอง
อัปเดต + ไฟล์แนบ

สาริตได้สด: แดชบอร์ดแยก 3 บทบาท → กดปุ่มพระจันทร์สลับโหมดมืด → ย่อหน้าต่างต่ำกว่า 992px ให้เมนูพับเป็นปุ่มขีดสามขีด → เปิดห้องแชทแล้วลากรูปมาวาง / กด Ctrl+V

ถาม: ถ้าผู้ใช้ปิด JavaScript ระบบยังใช้ได้ไหม?

ตอบ: **ส่วนใหญ่ยังใช้ได้ครับ** เพราะทุกฟอร์มเป็น HTML form จริง — ส่งข้อความ แนบไฟล์ กดปุ่มเปลี่ยนสถานะ ทำได้ปกติ สิ่งที่หายไปคือ สลับธีม, เมนูมือถือ, ลูกเล่นแชท, กล่องยืนยันก่อนลบ และ**การคลิกทั้งแถวในหน้ารายการ** เพราะเราใช้ `onclick` ที่แก้ `<tr>` — จุดสุดท้ายนี้ยอมรับว่าออกแบบผิด ควรเปลี่ยนเป็นลิงก์จริงเพื่อให้กด Tab ได้ด้วย

กระบวนการส่งมอบซอฟต์แวร์ของทีม

หัวใจของรายวิชานี้ไม่ใช่แค่ “เขียนโปรแกรมให้ทำงานได้” แต่คือกระบวนการที่ทำให้ทีม 4 คนส่งโค้ดเข้าที่เดียวกันได้โดยไม่พัง และตรวจสอบย้อนหลังได้ว่าใครทำอะไร

1 แยก branch

แต่ละคนเป็นเจ้าของ branch ของตัวเอง แยกจาก `main` ทำงาน แล้วส่งกลับผ่าน Pull Request ไม่มีใคร push ตรงเข้า

`main`

- `feature/models`
- `feature/auth`
- `feature/workflow`
- `feature/ui`
- งานร่วม: เทสต์ · CI · เอกสาร

2 Commit ที่อ่านรู้เรื่อง

ใช้รูปแบบ Conventional Commits ทำให้ประวัติอ่านออกว่าแต่ละครั้งทำอะไร

- `feat:` ฟีเจอร์ใหม่
- `fix:` แก้บั๊ก
- `style:` ปรับหน้าตา
- `test:` เพิ่มเทสต์
- `docs:` / `ci:` / `chore:`

3 Review ก่อน merge

ทุก Pull Request ต้องมีเพื่อนอย่างน้อย 1 คนกด Approve และ CI ต้องเขียวก่อนถึงจะ merge ได้

- มีแม่แบบ PR ให้กรอกครบทุกครั้ง
- ตั้ง Branch protection บน `main`
- ประวัติการรีวิวเห็นได้บน GitHub

4 ทดสอบอัตโนมัติ

GitHub Actions รันบน Ubuntu + Python 3.12 ทุกครั้งที่ push เข้า `main` และทุก Pull Request

- `manage.py check` — ตรวจสอบการตั้งค่า
- `makemigrations --check` — จับกรณีลืมสร้าง migration
- `manage.py test` — รัน 59 เคส

บันทึกเวอร์ชัน: ทุกการเปลี่ยนแปลงถูกบันทึกใน `CHANGELOG.md` ตามรูปแบบ Keep a Changelog แยกหัวข้อ “เพิ่ม / แก้ไข / เปลี่ยนแปลง” และมีเวอร์ชัน `1.0.0` ที่ระบุวันที่ปล่อยชัดเจน พร้อมส่วน `[Unreleased]` สำหรับงานที่เพิ่มหลังจากนั้น

ผลการทดสอบ — 59 เคส ผ่านทั้งหมด

ชุดทดสอบแยกตามหมวด

คลาสทดสอบ	เคส	ผู้ดูแล	ทดสอบอะไร
WorkflowTests	10	workflow	เส้นทางเต็มตั้งแต่แจ้งจนประเมิน · ตีกลับ · ข้ามขั้นไม่ได้
ImageUploadTests	10	UI/UX	แนบ/แสดง/ลบรูป · ปฏิเสธไฟล์ที่ไม่ใช่รูป
CommentThreadTests	8	ระบบสมาชิก	ห้องสนทนา · หมายเหตุภายในผู้แจ้งไม่เห็น
UnreadTests	7	workflow	ตัวนับข้อความค้างอ่าน · เปิดอ่านแล้วถูกล้าง
ModelTests	6	ฐานข้อมูล	ตรรกะบนโมเดล · โทมีไลน์ · ความสัมพันธ์ 1:1
AccessControlTests	5	ระบบสมาชิก	บังคับล็อกอิน · เปิดใบคนอื่นไม่ได้ · ได้ 403
ChatAttachmentTests	5	UI/UX	แนบรูป+เอกสาร · ปฏิเสธไฟล์อันตราย
ManageConsoleTests	4	workflow	คิวงาน · การงานช่าง · เพิ่มประเภทงาน
ActivityLogTests	4	workflow	ข้อความความคืบหน้าอัตโนมัติเมื่อสถานะเปลี่ยน
รวม	59	4 คน	ผ่านทั้งหมด · ไม่มีเคสใดล้มเหลว

ผลการรันจริง

```
$ python manage.py test
Creating test database...
.....
-----
Ran 59 tests
OK
System check identified no issues.
```

59 / 59 เคสผ่าน (100%) · เวลารันประมาณ 15-60 วินาทีแล้วแต่เครื่อง เพราะมี
เทสที่สร้างไฟล์รูปจริง

สิ่งที่ยังไม่ได้ทดสอบ (พูดตามตรง)

- ▶ ยังไม่มีเทสของหน้ารายงาน </report/>
- ▶ ยังไม่มีเทสของหน้าสมัครสมาชิกและหน้าเข้าสู่ระบบ
- ▶ ยังไม่ได้วัด code coverage ด้วย `coverage.py`

แผนการสาธิตระบบ – เดินครบ 1 วงจร ใน 14 นาที

เตรียมล่วงหน้า: รัน `python manage.py seed_data` แล้ว `python manage.py runserver` · เปิด

เบราว์เซอร์ 3 หน้าต่างแยกกัน ล็อกอินคนละบทบาท จะได้สลับได้ทันทีโดยไม่ต้องออกจากระบบ

#	เวลา	ผู้สาธิต	สิ่งที่ทำให้ดู
1	2 น.	ผู้แจ้ง <code>user1</code>	สมัครสมาชิกใหม่ 1 บัญชี → แจ้งซ่อมพร้อม แบบรูป 2 รูป → ดูโคม์ไลน์สถานะของใบตัวเอง
2	2 น.	ผู้ดูแล <code>admin</code>	เห็นตัวเลขแจ้งเตือนบนเมนู → เปิดคอนโซล <code>/manage/</code> → ติ๊กกลับ ใบหนึ่งพร้อมเหตุผล
3	1 น.	ผู้แจ้ง	เห็นสถานะ “ติ๊กกลับให้แก้ไข” พร้อมเหตุผล → แก้แล้วส่งใหม่ → กลับเป็น “รอตรวจสอบ” และเหตุผลถูกสร้างอัตโนมัติ
4	1 น.	ผู้ดูแล	มอบหมายช่าง → ชี้ให้ดูว่าระบบโพสต์ข้อความ “มอบหมายงานให้ช่าง...” ลงห้องสนทนาให้เอง
5	2 น.	ช่าง <code>tech1</code>	เห็นงานใหม่ในคิวของตน → กด รับงาน → บันทึกผลการซ่อม + แบบรูปงานเสร็จ → แจ้งงานเสร็จ
6	2 น.	ผู้ดูแล	เทียบ รูปก่อน-หลังซ่อม ในใบเดียวกัน → กด ตรวจรับผ่าน → ผู้แจ้งให้ 5 ดาว
7	2 น.	ผู้ดูแล	เปิด <code>/report/</code> ให้เห็นสรุปตามสถานะ ประเภทงาน ภาระงานช่าง และคะแนนเฉลี่ยที่เพิ่งเปลี่ยน
8	2 น.	ทุกคน	สลับ โหมดมืด · ย่อหน้าต่างให้เห็นว่ารองรับ จอมือถือ · รัน <code>python manage.py test</code> · สดให้เห็น Ran 59 tests – OK

ฉากสำรองถ้าเวลาเหลือ: ล็อกอินเป็น `user2` แล้วยิง URL ใบของ `user1` ตรง ๆ → ระบบแจ้งเตือน · ผู้ดูแลเขียน “หมายเหตุภายใน” แล้วสลับไปดูฝั่งผู้แจ้ง → มองไม่เห็น · ลองอัปโหลดไฟล์ `.exe` → ระบบปฏิเสธพร้อมบอกเหตุผล

สิ่งที่ยังทำไม่ได้ และแผนพัฒนาต่อ

เราเลือกที่จะพูดข้อจำกัดเองก่อนถูกถาม เพราะการรู้ว่าระบบของตัวเองยังขาดอะไร คือส่วนหนึ่งของการเป็นนักพัฒนา

พัฒนา

ข้อจำกัดที่รู้ตัว

- ▶ ยังใช้ **SQLite** และยังไม่ได้ deploy ขึ้นเซิร์ฟเวอร์จริง — รันที่ `127.0.0.1:8000` . `DEBUG` ยังเป็น `1` และ `SECRET_KEY` ยังเป็นค่า dev (โคัดอ่านจาก environment variable ไว้แล้ว เปลี่ยนได้ทันทีตอนขึ้นจริง)
- ▶ **เปลี่ยนช่วงกลางทางไม่ได้** — มอหมายได้เฉพาะใบที่ยังไม่เริ่มงาน และผู้ดูแลยกเลิกใบเองไม่ได้
- ▶ ฟิลด์ `admin_note` ถูกใช้ปน 3 ความหมาย (เหตุผลตกลับ / เหตุผลส่งกลับ / ความหมายเหตุตอนมอบหมาย) ควรแยกเป็นตารางประวัติ
- ▶ หน้ารายการ**ยังไม่มีการแบ่งหน้า (pagination)** — ถ้าใบแจ้งซ่อมเป็นพันใบจะโหลดช้า
- ▶ ยังไม่มี**แจ้งเตือนออกนอกกระบบ** (อีเมล/LINE) และยังไม่ได้วัด code coverage

แผนพัฒนาต่อ

- ▶ ย้ายไป **PostgreSQL** + ทำ **Docker Compose** ให้ทุกคนในทีมได้สภาพแวดล้อมเหมือนกัน
- ▶ **Deploy ขึ้นเซิร์ฟเวอร์จริง** พร้อม `DEBUG=0` , HTTPS และต่อ **CD** ให้ deploy อัตโนมัติเมื่อ merge เข้า `main`
- ▶ เพิ่ม **ระดับความเร่งด่วนและ SLA** พร้อมเตือนงานที่ค้างเกินกำหนด
- ▶ เพิ่ม **กราฟในหน้ารายงาน** เช่น จำนวนงานต่อเดือน และเวลาเฉลี่ยที่ใช้ซ่อมต่อประเภทงาน
- ▶ เพิ่มด้าน **coverage** และ **lint** ใน CI แล้วติด badge บน README
- ▶ ระบบ **เบ็ทอะไอร์แลนด์และคำใช้ง่าย** ผูกกับใบแจ้งซ่อมแต่ละใบ

ระบบใช้งานได้จริง และทีมทำงานร่วมกันได้จริง

9

ตารางฐานข้อมูล
ออกแบบและเชื่อมความสัมพันธ์เอง

3

บทบาทผู้ใช้
ตรวจสอบสิทธิ์ที่ฝั่งเซิร์ฟเวอร์ทุกครั้ง

7

สถานะงาน
ควบคุมด้วย state machine

59

เกสต์อัตโนมัติ
ผ่านทั้งหมด

สิ่งที่ทีมได้เรียนรู้จากโครงการนี้ ไม่ใช่แค่การเขียน Django แต่คือการแบ่งงานให้ไม่ทับกัน การรีวิวโค้ดของเพื่อนก่อน merge และการมีระบบทดสอบอัตโนมัติคอยดักความผิดพลาดแทนเรา — ซึ่งคือหัวใจของ DevOps